```
1  using CustomProjectRPG.Entities;
2  using CustomProjectRPG.Framework;
3  using System;
4  using System.Collections.Generic;
5  using System.Linq;
6  using System.Text;
7  using System.Threading.Tasks;
8  using CustomProjectRPG.Interfaces;
9
10 namespace CustomProjectRPG.Requests
11 {
12
13
14         public abstract class Request : IIdentifiable,
             ISerializeJSON<Request>
15         {
16             protected string _id;
17             protected string _title;
18             protected string _description;
19             protected int _apCost;
20             protected List<string> _requiredFlags;
21             protected List<string> _grantedFlags;
22
23             protected int _requiredFavor;
24             protected List<Reward> _rewards;
25
26             public string Id { get { return _id; } }
27             public string Title { get { return _title; } }
28             public string Description { get { return _description; } }
29             public int APCost { get { return _apCost; } }
30             public IReadOnlyList<string> RequiredFlags { get { return
                 _requiredFlags.AsReadOnly(); } }
31             public IReadOnlyList<string> GrantedFlags { get { return
                 _grantedFlags.AsReadOnly(); } }
32
33             public int RequiredFavor { get { return _requiredFavor; } }
34             public IReadOnlyList<Reward> Rewards { get { return
                 _rewards.AsReadOnly(); } }
35
36             protected Request(
37                 string id,
38                 string title,
39                 string description,
40                 int apCost,
41                 List<string> requiredFlags,
42                 List<string> grantedFlags,
43                 List<string> requiredClues,
44                 List<string> grantedClues,
45                 int requiredFavor,
```

```
46                  List<Reward> rewards)
47              {
48                  _id = id;
49                  _title = title;
50                  _description = description;
51                  _apCost = apCost;
52                  _requiredFlags = requiredFlags ?? new List<string>();
53                  _grantedFlags = grantedFlags ?? new List<string>();
54
55                  _requiredFavor = requiredFavor;
56                  _rewards = rewards ?? new List<Reward>();
57              }
58
59              public virtual bool CanOffer(PlayerEntity player)
60              {
61                  // Check flags, clues, and favor
62                  foreach (string flag in _requiredFlags)
63                  {
64                      if (!player.Memory.HasFlag(flag)) return false;
65                  }
66
67
68                  return player.Favor.CanUseFavor(player.Id, _requiredFavor);
69              }
70
71              public virtual bool CanComplete(PlayerEntity player)
72              {
73                  // Default: always completable if offered
74                  return true;
75              }
76
77              public virtual void ApplyRewards(PlayerEntity player)
78              {
79                  foreach (Reward reward in _rewards)
80                  {
81                      reward.Apply(player);
82                  }
83
84                  foreach (string flag in _grantedFlags)
85                  {
86                      player.Memory.AddFlag(flag);
87                  }
88
89                  foreach (string clue in _grantedClues)
90                  {
91                      player.Memory.AddClue(clue);
92                  }
93              }
94          }
```

```
95
96      }
97  }
98
```